BUSINESS PLAN - EDITION 2

AssetForge Business Plan

A content repurposing engine that runs on free software and one laptop

Founder: 【Your Name】

Edition: Second - supersedes the first plan written earlier today

Prepared: September 28, 2026

Status: v1.0.1 shipped - 24/24 tests green - dual-platform

Scope: Client repurposing service + WGU academic engine

Table of Contents

Executive Summary **1**

What Changed Since the First Plan **2**

The Problem: Long-Form Content Still Dies Young **3**

The Solution: What Was Actually Built **3**

The split-brain design, in plain English 5

The Windows bridge 5

How the founder uses the same machine 5

Build Status and Quality Evidence **6**

Revenue Model: Unchanged Prices, De-Risked Delivery **7**

Unit economics, now grounded in a real machine 7

What has to be true 8

Market and Competition **8**

Target Customers and Lead Generation **9**

The $0 lead engine: demo-first outreach 10

Protection: The Kitchen Is Now Literally Built **11**

Layer 1 - Legal foundations (low cost, early) 11

Layer 2 - The economic moat (the real defense) 12

Layer 3 - Relationship and switching costs 12

Layer 4 - The productized endgame 12

Setup and Daily Operations **12**

One-time setup on the Mac (when it arrives) 13

One-time setup on Windows (available now) 13

What daily operation looks like 13

The Academic Double-Duty **14**

Roadmap: Four Stages Done, Two Gates to Revenue **14**

Risks and Honest Answers: The Updated Register.....16

Appendix: Reference Card.....17

Note: This Table of Contents is generated via field codes. To ensure page number accuracy after editing, please right-click the TOC and select "Update Field."

Executive Summary

AssetForge is a done-for-you content repurposing service that runs entirely on free, open-source software installed on a single laptop. A client hands over long-form video or audio - a podcast episode, a property tour, a recorded webinar - and receives a monthly pack of ready-to-post material: social captions, posts, an article draft, quote graphics text, clip suggestions with exact timecodes, and an audio version of the show notes. The client does nothing except approve the results. There is no software bill, no cloud subscription, and no meaningful per-episode cost.

The first edition of this plan described what the machine would do. This edition describes what the machine does. As of today the entire production line exists as working software: version 1.0.1, eighty-five files, delivered as a single archive that installs with one command. Twenty-four of twenty-four automated tests pass and the code is lint-clean. The engine runs on two platforms - the founder's future Mac for client-grade speed, and the founder's current Windows laptop as an interim bridge - with the same pipeline, the same prompt library, and the same quality gates on both. The business is pre-revenue by one gate only: a final acceptance run on real hardware.

The launch focus is unchanged because nothing about it needed changing: real estate agents and small brokerages. Agents already record the raw material (listing walkthroughs, market updates, client testimonials), they are publicly reachable, and they already pay for lead generation every month. Their best content currently dies on one platform after a few days; AssetForge turns each video into weeks of consistent posting. Revenue arrives through monthly retainers priced at $297 for two episodes per month and $497 for four. Because each additional episode costs roughly nothing to process, the service stays profitable at prices well below traditional content agencies. This plan projects, as illustrative scenarios rather than promises, a path from the first client within roughly six weeks of hardware acceptance to roughly ten clients and $3,000 to $4,000 per month within six months of active outreach, using outreach that costs $0 to execute.

The same machine serves a second purpose that most competitors cannot copy: it is also the founder's study engine for a B.S. in AI Engineering at Western Governors University. That engine is no longer a design goal - it shipped with v1.0.1. Lecture audio enters the same intake as client video, a local knowledge base stores the course material, and the study command returns answers that cite their sources in that material. One machine, two outputs, one cost basis of effectively zero.

What Changed Since the First Plan

The first edition of this plan was written when the project consisted of an approved requirements document and an empty repository scaffold. This section is the delta: what the first plan promised, what actually got delivered, and what those changes mean for every claim that follows. The short version is that the engineering risk has been retired - the machine exists and is tested - while the business model, the pricing, and the launch niche are deliberately unchanged.

Table 1: First plan versus current reality

What the first plan said	Where the project stands now	Why it changed
Product status: a pipeline to be built in Stage 1	Built and shipped as v1.0.1 (85 files); 24 of 24 automated tests pass; lint clean	Build phases 3 through 7 of the founder's process executed in full
Architecture: one automation tool (n8n) as the spine	Split-brain design: a host daemon owns every heavy stage; n8n handles fan-out only and can go down without stopping delivery	Requirements Amendment 2 resolved the single-point-of-failure risk before any code was written
Platform: runs on the founder's Mac laptop	Dual-platform: full macOS build plus a Windows bridge with a swappable transcription engine	The MacBook had not arrived; the founder was on a Windows laptop, so the project continued instead of stalling
Quality: provenance checking as a design goal	Provenance is a ship gate: any asset the machine cannot trace to its source is flagged, never silently passed	Locked as decision D4 in the engineering plan-of-record; automated tests enforce it
Budget: $0 software spend (planned)	$0 spent (actual)	The free-stack blueprint held up under real implementation
Roadmap: six stages, none started	Stages 0 through 3 complete; Stage 1 hardware acceptance is the current gate	The build compressed a week-by-week schedule into a single working day of AI-assisted execution

Two things deserve equal weight in this delta. First, what did not change: the prices, the beachhead niche, the $0 constraint, the demo-first sales motion, and the honesty sections are all untouched, because none of them were the bottleneck. Second, what the changes cost: the original plan assumed the Mac would be in hand for all engineering; in reality the hardware had not shipped, so a Windows path was built to keep moving. That detour produced a genuine asset - the engine now runs on two platforms and can survive a dead laptop - but it

also added an honest caveat that appears throughout this edition: client-grade turnaround speed is currently a Mac capability, pending that machine's arrival.

The Problem: Long-Form Content Still Dies Young

Nothing about the market problem changed while the machine was being built, and that is the point of restating it: the gap the first plan targeted is the gap the finished engine now addresses. Professionals who market themselves with video face a structural mismatch - their most valuable material is long-form, but every audience they want to reach lives on short-form platforms. A two-hour podcast or a fifteen-minute property tour contains enough insight for a month of posts, but extracting it by hand takes six to ten hours per episode. Almost nobody sustains that effort. The content gets published once, seen by whoever sees it that day, and effectively dies.

The existing alternatives each fail a different way, exactly as described in the first plan. Do-it-yourself AI tools in the roughly $20 to $80 per month range still require the user to learn the tool, sit through exports, and make every creative decision themselves; the work moves, it does not disappear. Human content agencies charge roughly $500 to $2,000 per month, deliver on the timescale of days, and produce generic output that ignores local context. Offshore virtual assistants cost less but introduce quality variance, training overhead, and churn.

The result is a consistent, well-documented gap: nearly everyone producing long-form content agrees that repurposing matters, and almost nobody actually does it consistently. The value is agreed upon; the labor is the bottleneck. A built, tested machine that removes the labor at a cost basis of effectively zero is now positioned to close that gap, and the remainder of this plan describes that machine as it actually exists rather than as it was sketched.

The Solution: What Was Actually Built

In plain terms, AssetForge behaves like a one-person media team that works overnight for free. The client never sees a tool, a login, or a settings screen; from the client's point of view the entire service is: send the video, receive a folder of finished content, approve it, watch it go out on schedule. That description was the design brief, and it is now a description of shipped software. The production line runs as seven ordered stages, each of which exists in the delivered archive:

1. Fetch: a dropped file or link enters the intake bus. Every file follows a strict naming contract - client, source, date, slug - so nothing is ever misfiled and every downstream stage knows whose content it is holding.

2. Normalize: whatever format arrives is standardized into the working formats the rest of the line expects.
3. Clean: music beds are detected and separated so they do not pollute transcripts, and background noise is reduced before transcription.
4. Transcribe: every word is captured with timestamps. On the Mac this uses Apple's Metal-optimized engine; on Windows it uses a swappable engine (faster-whisper) that runs on ordinary CPUs or NVIDIA graphics cards. The transcript honestly records which engine ran, so provenance never lies about its origins.
5. Segment: the raw word stream is rebuilt into sentences and paragraphs with timing attached, the structure every later stage depends on.
6. Generate: local writing models draft the asset pack - ten asset types - using a prompt library that is versioned by content hash, so every output can always be traced to the exact prompt that produced it.
7. Package: everything lands in a clearly organized client folder with a manifest recording every file, its purpose, and the part of the source it came from. Posts are staged as drafts in the scheduling tool, and the client folder contains no hints of the machine's internal paths.

Table 2: What is inside a standard asset pack

Asset	Purpose
Show notes	Episode summary for the client's website or newsletter
Article draft	A 600 to 1,000 word blog post drawn from the episode
Social captions	Instagram and Facebook captions in the client's voice
Short-form posts	A five to eight post thread plus a LinkedIn post
Quote set	Eight standout lines ready to be turned into graphics
Clip list	Five recommended clips with exact start and end timecodes
SEO block	Title and description variants for video platforms
Audio notes	A narrated audio version of the show notes

Every asset carries provenance: the manifest records which span of the source it was derived from. This is a quality-control feature borrowed from research-grade systems - if the machine cannot support a claim from the source material, the output is flagged rather than

guessed. It is the difference between a service that occasionally embarrasses a client and one that does not, and it is enforced by the pipeline itself rather than by founder vigilance.

The split-brain design, in plain English

The build's most important architectural decision is invisible to the client: the system is deliberately split in two. A host daemon - a small always-on program on the laptop - owns every stage that touches audio, video, or the writing models, because those stages need the laptop's specialized hardware. The free automation tool (n8n) handles only notifications and fan-out work. The practical consequence: if the automation layer crashes, gets deprecated, or is simply switched off, packs still get produced and delivered. The convenient parts of the system are optional; the money-making part is not. Jobs are also recoverable - the system keeps a ledger of every job and every stage, so a failed run resumes from the failed stage instead of starting over.

The Windows bridge

Because the MacBook had not arrived, the build was extended to run on the founder's current Windows laptop rather than pausing the project. The transcription engine is a configuration switch, not a code change - one setting swaps Apple's Metal engine for faster-whisper, and the system auto-detects NVIDIA graphics cards when present. The honest trade is speed: on the Windows machine, a two-hour episode takes roughly two to five hours on a typical CPU (an overnight, unattended run) versus the Mac's acceptance target of under thirty minutes; a laptop with an NVIDIA card lands between the two. Client-grade same-day turnaround therefore remains a Mac capability for now. The Windows bridge keeps the machinery, the academic engine, and the demo-pack pipeline moving until the hardware arrives, and nothing built for Windows is wasted work - it is the same pipeline, and both platforms share the ledger format, the prompts, and the quality gates.

How the founder uses the same machine

The founder runs three jobs on one engine. First, the client service described above. Second, the founder's own social presence: the machine processes the founder's content the same way, which doubles as public proof of work - every post the founder publishes is an advertisement for the service that produced it. Third, the academic engine: lecture audio enters the same intake, and instead of social captions the machine produces study notes and cited answers from a local knowledge base. The answer to whether a business can coexist with a degree program is built into the architecture itself.

Build Status and Quality Evidence

A plan that says the machine works without offering evidence is a brochure. This section is the evidence: what was delivered, what was verified, and - just as important - what has not yet been proven on real hardware. The distinction matters because the automated tests prove the logic of the pipeline, while the acceptance numbers in the requirements document (processing speed and provenance pass rate on a real two-hour episode) still require a physical run on the target machine.

Table 3: Build status at a glance (as of this edition)

Item	Status
Delivered artifact	assetforge-v1.0.1.zip - 85 files, single-layer extraction, installs with one command on either platform
Automated tests	24 of 24 passing; coverage includes the naming contract, the ledger, audio stitching, provenance extraction, manifest scrubbing, and engine-honesty checks
Code quality	Lint-clean under ruff; compiles clean; test gate runs automatically as part of setup
Pipeline modules	Roughly 20 modules across intake, processing, generation, packaging, notification, and academic subsystems
Prompt library	11 prompt files, versioned by content hash; the exact prompt version is recorded in every client manifest
Engineering record	Requirements document with two amendments; plan-of-record with 8 ranked unknowns and 6 locked decisions; UI-less design note; Windows bridge guide

The case for trusting this build is strongest where it embarrassed itself first: the automated tests caught three genuine defects before any client could ever see output. The transcription stitcher was treating overlapping audio chunks as duplicates and silently dropping content; the provenance extractor was double-counting overlapping quotations, inflating the trust score of assets that had not earned it; and the client-facing manifest was leaking the machine's internal folder paths, a small but real privacy defect. Each bug was fixed and each now has a regression test standing guard over it. This is the difference between claiming quality and having a machine that enforces it - and it is the reason the acceptance numbers that follow are worth measuring rather than assuming.

What remains unproven is specific and bounded. Two numbers from the requirements document are still targets rather than results: that a real two-hour episode processes in under thirty unattended minutes, and that at least 95 percent of generated assets pass the provenance gate. Both are Stage 1 acceptance criteria, and both require the founder to run one real episode on real hardware - the Mac for the headline numbers, or the Windows bridge for a slower but honest baseline. Until that run happens, every performance statement in this plan should be read as a design target backed by tested logic, not as a measured result.

Revenue Model: Unchanged Prices, De-Risked Delivery

The pricing structure is deliberately unchanged from the first plan, because the first plan's pricing was built on a cost basis that the finished engine now confirms rather than assumes. There is no free tier for ongoing work, but every prospective client receives one complete sample pack built from their own publicly posted video at no charge. The demo-first approach exists because the product is difficult to describe and effortless to demonstrate: a finished pack made from the prospect's own content closes the imagination gap that a sales page cannot.

Table 4: Pricing structure (unchanged)

Plan	Price	What the client receives
Pilot pack	$0 (one time)	One full asset pack from one video; the demonstration that sells the retainer
Starter	$297 / month	Two episodes per month, full pack each, scheduled drafts, email delivery
Growth	$497 / month	Four episodes per month, priority turnaround, quarterly content review call
Brokerage	Custom	Per-agent volume pricing for teams of five or more agents

Unit economics, now grounded in a real machine

Each episode consumes roughly fifteen minutes of founder review plus unattended machine time: under thirty minutes per two-hour episode on the Mac (the acceptance target), or an overnight run on the Windows bridge. There is no software cost, no API cost, and no infrastructure cost - the $0 budget survived contact with implementation. On the Starter plan, a paying client represents roughly one hour of human attention per month in exchange for $297, and the gross margin on every plan exceeds 90 percent because the only true inputs are

electricity and attention. The one honest qualifier is timing: same-day turnaround is a Mac capability until the Windows bridge is retired or augmented, so early clients are served from whichever machine passes acceptance first.

Table 5: Illustrative revenue scenarios (planning assumptions, not forecasts)

Period	Clients (assumed)	Monthly revenue	Primary activity
Months 1-2 after acceptance	1 to 3	$297 to $1,191	Demo-first outreach, first packs delivered at client-grade speed
Months 3-6	6 to 10	$1,782 to $4,000	Referral engine, brokerage conversations
Months 6-12	10 to 15	$3,000 to $6,000	Retention, broker deals, process polish
Year 2 option	n/a	Subscription revenue	Productized self-serve version of the engine

What has to be true

The honesty section, updated rather than replaced. First, revenue tracks outreach volume almost linearly: five sample packs sent per week is the engine that makes the first column of Table 5 happen, and cutting outreach to zero cuts revenue to zero. Second, churn is real - clients pause in slow markets, and retention depends on demonstrable consistency, not on the novelty of the first pack. Third, a solo operator with a full course load has a practical ceiling of roughly fifteen to twenty clients before service quality or coursework suffers; the Year 2 productized option exists to break that ceiling. Fourth, the scenarios assume the social platforms continue allowing scheduled posting through connected accounts, which the design treats as a convenience rather than a dependency - if an API tightens, delivery falls back to ready-to-paste drafts and the retainer value survives. Fifth, and new since the first plan: client-grade turnaround waits on hardware. The acceptance run is the gate between the current state and the first sample pack a prospect receives; the Windows bridge keeps every other part of the business moving in the meantime.

Market and Competition

Repurposing is a proven, crowded category on the software side and a fragmented, inconsistent category on the service side. That combination remains the opportunity: the software proves demand exists, and the service gap proves the demand is not being met for people who do not want to become video editors. AssetForge does not compete on having better AI - the underlying models are the same open ones everyone can access. It competes

on absorbing all of the work at a price point the software vendors cannot touch and the agencies cannot match without losing money. What changed since the first plan is that this positioning is now backed by a shipped, tested machine rather than a cost model in a spreadsheet.

Table 6: Competitive landscape (unchanged structure, current framing)

Alternative	Typical cost	What the buyer gets	Where it falls short
DIY AI tools (Opus Clip, Descript, Captions class)	$20 to $80 per month	Clips and drafts the user assembles themselves	The user still does the work; output is generic; per-seat costs stack
Content agencies	$500 to $2,000 per month	Done-for-you content	Days of turnaround; minimal niche awareness; high overhead
Virtual assistants	$500 to $1,500 per month	Human hours	Slow, variable quality, training and churn overhead
AssetForge	$297 to $497 per month	Done-for-you packs, niche-tuned output, provenance-checked quality	Solo capacity ceiling until the productized version ships; same-day turnaround gated on Mac hardware

The defensible position is the cost structure, and two assets have hardened since the first plan. The prompt library - the accumulated instructions that make output niche-appropriate - is versioned by content hash inside every manifest, so the business always knows exactly which prompt version produced which asset; it is a trade secret that improves with every client and is never exposed to the people who receive its output. The provenance discipline is the second: every quotation traceable to a span of source material, and failures flagged rather than hidden. Generic DIY tools structurally cannot offer that guarantee, because their business model depends on the user not checking. In a market where buyers cannot distinguish model quality but can absolutely distinguish turnaround time, monthly price, and whether a quote embarrasses them, that asymmetry is the strategy.

Target Customers and Lead Generation

The launch niche is chosen for reachability as much as need, and the build did nothing to change the choice. Real estate agents satisfy four conditions at once: they already produce video (the raw material exists without education), they are locally concentrated and publicly listed (every agent's contact information is an advertisement), they already spend on

visibility (lead-generation platforms, mailers, paid ads), and their content has an unusually short shelf life relative to how much of it they produce. The secondary segments below are activated later from the same engine with only prompt and outreach changes - the niche is a configuration, not a rebuild.

Table 7: Target segments in activation order

Segment	Their pain	Where to find them
Real estate agents (launch)	Listing videos die in a week; no time to post consistently; already pay for leads	YouTube local listing searches, Instagram local hashtags, brokerage Facebook groups, public agent directories
Podcasters (second)	Episodes vanish after publish day; they already know repurposing matters	Podcast charts by category, podcasting communities, LinkedIn
Coaches and consultants (third)	Webinars and long videos with no derivative funnel	LinkedIn, Instagram, YouTube comments on their own long-form
Local service businesses (bundle)	No content operation at all; need automation before content	Local Facebook groups, chambers of commerce, referrals

The $0 lead engine: demo-first outreach

Because a sample pack costs roughly nothing to produce, the outreach model leads with finished work instead of promises. The weekly rhythm: the founder selects five local agents who have recently posted listing or market-update videos, runs each video through the machine, and sends the completed pack directly to the agent with a two-line note - the pack is theirs to keep regardless, and if they want one every week, the retainer details are attached. One discipline is new since the first plan: outreach does not begin until the Stage 1 acceptance run passes, because the first sample pack a prospect receives must be produced at client-grade speed and quality on production hardware. A demo produced on a slow bridge machine would demonstrate the wrong thing.

Table 8: Weekly outreach rhythm and assumed funnel

Activity	Volume	Expected result (assumption)
Sample packs produced and sent	5 per week	Roughly 20 per month at $0 cost
Replies and conversations	10 to 20 percent	Two to four genuine conversations per month

Activity	Volume	Expected result (assumption)
Closes from conversations	25 to 40 percent	One to two new clients per month
Referrals (one free month per intro)	Ongoing	Compounding, zero-cost channel after month two

Two accelerants sit behind the weekly rhythm, and both are stronger than they were in the first plan. The brokerage angle: one conversation with a managing broker can convert into a multi-agent deal worth five to twenty individual clients in a single sale. And the proof-of-work angle: the founder's own feeds, powered by the same shipped engine, function as a permanently running demonstration - a prospect who watches the founder post consistently for a month has already seen the product work, and the machine producing those posts is the same one that would produce theirs.

Protection: The Kitchen Is Now Literally Built

The honest answer first: the individual components are free and public, and no patent or license stops a determined, technical person from assembling something similar. Protection therefore does not come from pretending the recipe is secret magic. It comes from four layers, each doing a different job - and the first plan's central claim in this section has literally come true in the interim: the kitchen is no longer a blueprint. It is eighty-five files of working, tested machinery.

Layer 1 - Legal foundations (low cost, early)

The custom code, prompt library, and workflow configurations are original work protected by copyright automatically upon creation; registration with the U.S. Copyright Office (currently under $100 per work) is available if it ever becomes worth formalizing. The engine stays in a private repository and is never shown to clients - they receive outputs, not internals, which also preserves its trade-secret status. The prompt library gained a structural improvement during the build: it is versioned by content hash, and every client manifest records the exact prompt version that produced each asset. That record is simultaneously a quality trail (any output can be traced to its instructions) and trade-secret hygiene (the library lives only on the founder's machines). Every client signs a service agreement containing confidentiality and non-solicitation clauses (free templates from SCORE and equivalent small-business resources cover this; a reviewed draft is scheduled in a later build phase). The brand name and logo can be registered as a federal trademark (roughly $350 per class) once revenue justifies it - not before, because a trademark on a pre-revenue name buys nothing.

Layer 2 - The economic moat (the real defense)

A copycat must replicate the cost structure to compete on price, and most cannot. Agencies pay per generation, per minute of transcription, and per seat for scheduling software; AssetForge pays effectively nothing per episode, and that figure is now an operating fact rather than a projection. The business can undercut any competitor and remain profitable, outlast them in a price war, and give away unlimited sample packs as an acquisition strategy that would be a loss-maker for anyone paying retail software rates. There is also a new, blunter version of the first-mover point: a competitor starting today is starting from an empty repository, while the founder starts from tested, dual-platform, documented machinery. In software businesses, being one build ahead is ordinary; in service businesses, being one build ahead while charging service prices is decisive.

Layer 3 - Relationship and switching costs

Once a client's posting rhythm, brand voice, and content calendar live inside the service, leaving means rebuilding all of it elsewhere. Retention is the quiet defense: consistent weekly output, provenance-checked quality, and a reporting cadence make the service part of the client's operations rather than a vendor they re-evaluate monthly.

Layer 4 - The productized endgame

When the engine becomes a self-serve product in Year 2, the code runs on servers and is never distributed to customers, which changes the theft question entirely: there is nothing to copy from the outside. By that point the brand, the client base, and the data flywheel (what works per niche) are the assets, and none of them are in the code.

Verdict, plainly, and updated for the current state: someone can steal the recipe; they cannot steal the kitchen that is now actually built, the customers already eating, or the cost basis that lets the founder wait them out. In a service business, speed and consistency beat secrecy - and the speed is now shipped.

Setup and Daily Operations

The founder has exactly two obligations: one setup session of roughly sixty minutes on whichever machine is being pressed into service, and about fifteen minutes of review per client episode thereafter. Everything else - fetching, cleaning, transcribing, drafting, packaging, staging, notifying - happens without human involvement. The shipped system includes a command-line interface (submit, watch, status, resume, and related commands) that gives the founder a single handle for the entire machine, and every service auto-restarts if the laptop reboots.

One-time setup on the Mac (when it arrives)

1. Unzip the delivered archive and run the one-line setup command. It installs every dependency, creates every folder, downloads the AI models, installs the always-on daemon, and finishes by running the full 24-test gate automatically (roughly 20 minutes, mostly waiting).
2. Create four free accounts: GitHub (private code storage), Tailscale (secure access from phone and watch), Resend (client notification email), and Docker Hub (the automation container).
3. Tap two links on the iPhone to install the intake Shortcuts; the Apple Watch inherits the same capability the day it arrives - the recipe is already written.
4. Connect the client's social accounts once through the scheduling tool's standard login flow.

One-time setup on Windows (available now)

1. Run the single PowerShell setup script. It installs the engine's dependencies (including the local model runtime, audio tooling, and the downloader), automatically picks a writing model that fits the laptop's memory, configures the environment, and runs the same 24-test gate as the Mac path.
2. The same four free accounts as above - they are shared across machines, not per-platform.
3. The same iPhone Shortcuts work unchanged; the intake folder is what matters, not which machine hosts it.
4. No client social accounts are connected on the interim machine - the Windows bridge serves the founder's own content and the academic engine until the Mac arrives.

What daily operation looks like

From the phone: the founder speaks a note or shares a link, and the file lands in the intake folder from anywhere. From the laptop: drag a file into a folder. That is the entire operator interface. Jobs run serialized overnight when the machine is otherwise idle; the founder wakes to finished packs, spends fifteen minutes per pack approving, and the scheduling tool handles publication. There is no daily maintenance task list - the system was designed so that the founder's scarce resource, attention, is spent only on the two things machines cannot do: judgment on output quality and conversations with clients.

The Academic Double-Duty

A standard objection to running a business during a degree program is that the two compete for the same hours. This build inverts the objection, and the inversion is no longer theoretical: the academic engine shipped inside v1.0.1. Lecture audio rides the same intake bus as client video, a local knowledge base stores the course material, and the study command returns answers that cite their sources in that material - the same provenance discipline applied to clients, pointed at coursework. The pipeline that cuts client videos into timestamped chunks is a data-structures exercise; the searchable knowledge base is a database-design exercise; the quality harness that scores output against source material is a data-analysis exercise. The B.S. in AI Engineering at WGU is competency-based, which rewards demonstrated skill - and the business now produces demonstrated skill as a byproduct of existing.

The practical effect is that the hours the business costs are substantially returned by the hours it saves. The founder studies from material the machine prepared - study notes and flashcard decks aligned to the current course - and the audio versions convert commute time and gym time into review time. The same machine bills clients by day and accelerates the degree by night, and the Windows bridge means this half of the double-duty never waits on the Mac: study output is not speed-sensitive the way client turnaround is.

Roadmap: Four Stages Done, Two Gates to Revenue

The original roadmap budgeted six stages across roughly six weeks. The build portion of that roadmap - stages that produce working systems rather than reports - completed in a single working day of AI-assisted execution, and the table below reflects the current state of each stage honestly, including the two that remain. The pattern to notice: nothing between the project and its first paying client is engineering. What remains is one hardware acceptance session and the outreach calendar.

Table 9: Staged roadmap, current status

Stage	Status	Exit criteria
Stage 0	COMPLETE	Repository, folder structure, secure phone-to-laptop link, setup script verified end to end

Stage	Status	Exit criteria
Stage 1	BUILT - ACCEPTANCE PENDING	One command produces a complete asset pack; acceptance means one real two-hour episode processed in under 30 unattended minutes with provenance pass rate at or above 95 percent, on production hardware
Stage 2	BUILT - DELIBERATELY GATED OFF	Packs flow into the scheduling tool as drafts; the automation containers exist in the archive and switch on with one command after acceptance passes
Stage 3	COMPLETE	Academic intake live: lecture audio produces study material and cited answers; the engine eats its own coursework
Stage 4	NOT STARTED	Outreach machine running: five sample packs per week; intake form, booking page, and CRM operating; first paying retainer
Stage 5	FUTURE	Productized self-serve version scoped from real client usage patterns

The founder's next concrete action is a single working session, spelled out here so it cannot hide: unzip the v1.0.1 archive, run the setup command, run the smoke test, drop one real two-hour episode into the intake folder, and read the numbers the run produces - total time and provenance pass rate. On the Mac, passing means the client-grade engine is live. On Windows, the same session produces an honest baseline and keeps the academic engine and the founder's own content running until the Mac arrives. The second action is scheduled around it: a full security audit (phase 8 of the build process) runs before the first client is onboarded, so that the first external person to touch the system touches an audited one.

Each stage in the table still ends with a working system rather than a report, no stage depends on spending money, and the first revenue milestone - one paying retainer - now sits behind one session of hardware time plus the outreach calendar, not behind any remaining engineering.

Risks and Honest Answers: The Updated Register

The risk register changed shape since the first plan, and the change is informative: every risk of the form "can this be built" is gone, because it is built. What remains are risks of execution, timing, and markets - the risks of a business rather than the risks of a project. The register below reflects that shift, with the hardware-timing risks new since the first edition.

Table 10: Updated risk register

Risk	Severity	Mitigation
Outreach volume drops (degree workload spikes)	High	Sample packs are produced in batch on weekends; the pipeline runs itself; outreach is the only irreplaceable human hour
MacBook delivery slips further	Medium	The Windows bridge keeps the academic engine, the founder's own content, and demo-pack production moving; only client-grade speed waits
On-device acceptance misses its targets (over 30 minutes, or provenance under 95 percent)	Medium	The gate exists precisely to catch this before a client does; tuning options are model size, chunking, and prompt scope; the requirements document forbids outreach until the numbers are real
Windows slow-lane ceiling (hours per episode on CPU)	Medium	Treat the bridge as machinery, study, and demo capacity - not same-day turnaround; the Mac restores service-grade speed on arrival
Social platform API friction tightens	Medium	Approve-first draft flow everywhere; delivery never depends on auto-posting; packs remain ready-to-paste
Client churn in slow market	Medium	Reporting shows delivered value; pause-friendly terms beat cancellation; referral free month keeps advocates warm
A funded competitor undercuts on price	Medium	Cost basis near zero lets the business match any price profitably; the hash-versioned prompt library and provenance discipline are not price features

Risk	Severity	Mitigation
AI output quality incident	Medium	Provenance ship gate on every asset; automated regression tests; human approval before anything ships; failures logged and fed back into prompts
Work split across two machines during the bridge period	Low	Both platforms share the ledger format, prompts, and contract grammar; the repository is the single source of truth; the Mac becomes production on acceptance
Single-machine failure	Low	Full backups plus code in GitHub; rebuild is one script and one model download

Two risks deserve emphasis beyond the table, unchanged from the first plan because they are structural. The first is founder attention: the revenue scenarios assume roughly five outreach hours per week, and during exam periods that assumption fails - the mitigation is batching and accepting a slower month rather than a hollow one. The second is the temptation to widen scope: adding niches, custom deliverables, and one-off projects before the core retainer is proven is the most common way early service businesses die, and the roadmap deliberately sequences expansion only after real client usage has been observed. One deliberate omission from this edition's table: the voice-cloning question from the first plan is settled (cloning is skipped in v1), the decision stands, and it no longer needs a row.

Appendix: Reference Card

This section exists so the plan can be found again without searching. It records where the working documents live, which artifacts are current, and what the next checkpoint in the build process is. This second edition supersedes the plan written earlier the same day; the first edition remains on disk for comparison.

Table 11: Project references

Item	Value
Conversation platform	zai-web chat
Conversation (chat) ID	4424fecf-bcc7-4ed9-b703-2fa848dc2f73
Session ID	web-3c5c7fbf-76d9-4192-9427-b8a100681d66
Plan edition	Second edition, September 28, 2026 - current-state revision

Item	Value
Current build artifact	download/assetforge-v1.0.1.zip (85 files, dual-platform)
Requirements document	download/prd-assetforge-v1.md (approved, Amendments 1 and 2 applied)
Engineering plan-of-record	docs/plan-phase3.md inside the archive (8 ranked unknowns, 6 locked decisions)
Windows bridge guide	docs/windows-bridge.md inside the archive (swap table, RAM-tier expectations, tonight checklist)
Free-stack blueprint	download/free-stack-blueprint.md
First plan edition (superseded)	download/AssetForge-Business-Plan.docx
Account email	【Add from account settings - the architect cannot read account credentials】
Founder name	【Your Name】

Next checkpoint in the build process: the Stage 1 on-device acceptance run (founder action - one real episode, two numbers: time and provenance pass rate), followed by the phase 8 security audit, followed by the Stage 2 automation flip. The outreach machine starts only after acceptance, per the discipline in the lead generation section. Note on the conversation link, carried forward from the first edition: chat platforms generate a shareable link through their own share function rather than exposing one inside the session; the identifiers above are sufficient to locate this session in support contexts.